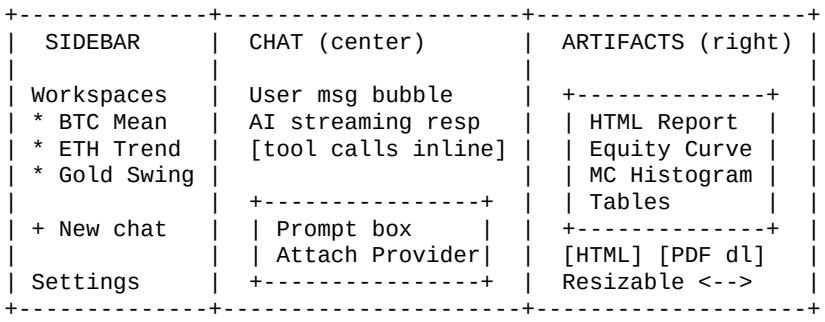
StratForge AI

Final Locked Plan - Desktop Backtesting App

1. Aapke 8 Decisions (Locked)

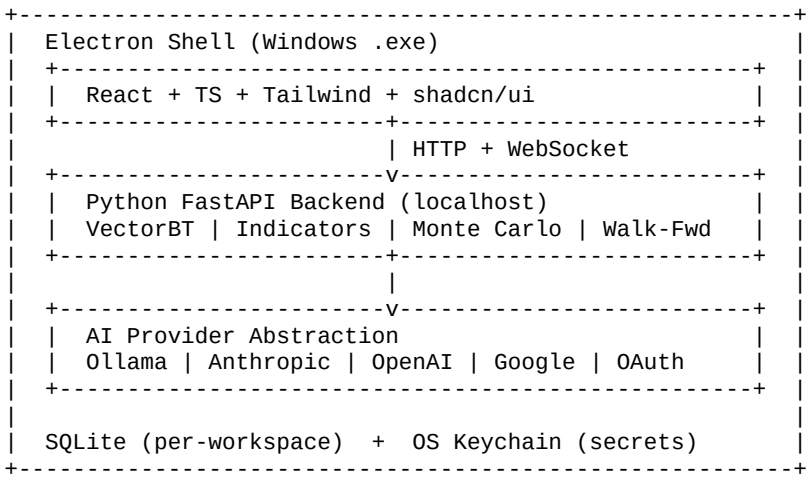
#	Decision	Status
1	OS scope	Windows first, Mac/Linux v2
2	Claude CLI OAuth	Claude subscription login + API key alternative
3	Local LLM	Ollama only
4	Report format	HTML + PDF, right-side Artifacts panel
5	Offline mode	Auto-supported via Ollama (no separate toggle)
6	App name	StratForge AI
7	Distribution	.exe installer + license system + auto-update
8	Workspace memory	Auto-generated per workspace, loads in future sessions

2. Final UI Layout (Claude Code style)



3. Architecture Overview

Electron shell hosts a React UI; a bundled Python FastAPI process handles all heavy computation (VectorBT engine, indicators, Monte Carlo). AI providers are abstracted behind a single interface. Each workspace is an isolated folder with its own database, data, and memory.



4. Workspace + Memory System

Har workspace ek isolated folder hai. Conversations se memory auto-generate hoti hai aur next session me context ke roop me inject hoti hai - Claude Code's own memory model ki tarah.

```
~/StratForge/workspaces/<name>/
|-- workspace.json          # config, default provider
|-- data/                   # uploaded CSVs
|-- reports/                # generated HTML + PDF
|-- chats.db                # SQLite conversations
|-- memory/                 # AUTO-generated
    |-- MEMORY.md           # index
    |-- preferences.md      # user's trading style, risk tolerance
    |-- data_context.md     # assets, timeframes used
    |-- strategies_tried.md # kya kaam kiya, kya fail hua
    |-- decisions.md        # user's explicit choices
```

Memory Flow

1. Chat start - workspace memory context me inject hota hai (AI remembers last session).

2. Chat end / idle - background summarizer chalta hai, durable facts extract karke memory files update karta hai.

3. Auto-classify - summarizer decide karta hai fact kaunsi memory file me jayega.

4. User override - user manually memory edit ya delete kar sake.

5. AI Orchestration (Tool Calling)

AI directly compute nahi karta. Wo decide karta hai kaunsa tool kis order me chalana hai - Python backend actual work karta hai. Yehi token-saving principle (95% code, 5% AI) ka core hai.

```
User: "BTC 1h data pe RSI+BB mean-reversion, last 2Y, optimize karo"
   |
AI tool calls (sequence):
  1. load_data(file="BTCUSDT_1h.csv", range="2y")
  2. calculate_indicators(["RSI", "BBANDS"])
  3. build_strategy(type="mean_reversion", entry="RSI<30 & price<BB_lower")
  4. run_backtest(fees=0.1%, slippage=0.05%)
  5. optimize_parameters(grid={...}, top_n=5)
  6. validate(walk_forward=True, monte_carlo=1000)
  7. generate_report(format=["html","pdf"])
   |
Chat: "Strategy score 87/100, confidence 92%. Report right side me open hai."
Artifacts panel: report auto-open
```

6. Final Tech Stack (LOCKED)

Layer	Choice
Shell	Electron 30+
UI	React 18 + TypeScript + Tailwind + shadcn/ui + Zustand
Backend	Python 3.11 + FastAPI + Uvicorn
Engine	VectorBT + Pandas + NumPy + TA-Lib
Storage	SQLite per-workspace + OS keychain (keytar)
Streaming	WebSocket (token-by-token)
AI SDKs	Anthropic SDK, OpenAI SDK, Google GenAI, Ollama HTTP
OAuth	Loopback redirect + PKCE (system browser)
PDF export	Playwright (HTML to PDF)
Packaging	electron-builder + PyInstaller
Updater	electron-updater (GitHub Releases or S3)
License	TBD (see mini-questions)

7. Build Plan - 10 Phases (~10 weeks)

Phase	Scope	Time
P1	Electron + Python shell + 3-pane Claude Code UI skeleton	1 wk
P2	Workspace system + SQLite + sidebar navigation	1 wk
P3	CSV/Excel upload + data preview + 70+ indicator engine	1 wk
P4	Provider Manager: Ollama + Anthropic/OpenAI/Gemini API keys	1 wk
P5	Claude Code subscription OAuth + token refresh	1 wk
P6	AI Orchestrator + tool calling + streaming chat	1 wk
P7	Backtest + batch optimize + walk-forward + Monte Carlo	1.5 wk
P8	Artifacts panel + HTML/PDF report generator	1 wk
P9	Per-workspace memory subsystem (summarizer + injection)	1 wk
P10	License system + auto-update + installer + code signing	1 wk

8. Final 3 Mini-Questions (Pending)

Plan 100% lock karne ke liye ye 3 chhote decisions reh gaye hain:

Q1. License server hosting

Aap khud host karoge (VPS: DigitalOcean / Hetzner), ya **Keygen.sh / Paddle / Lemon Squeezy** use karein? Third-party me built-in payments + less ops; self-hosted me full control + no recurring SaaS cost.

Q2. Memory summarizer model

Background memory summarizer user ka selected AI use kare (user ke tokens kharcha honge), ya hamesha ek chota sasta model (jaise Claude Haiku / local Llama) background me chale (cost predictable, user tokens safe)?

Q3. Pricing tiers

Option A: Free (local LLM only) + Pro ($X/mo all providers + auto-update) + Team (multi-machine).
Option B: Single paid tier with free trial.
Option C: Lifetime license, one-time purchase.

Ye 3 answers milte hi plan 100% locked ho jayega aur Phase 1 (Electron shell) build start hoga. Ab tak koi code nahi likha - sirf planning + memory files.